

UCUENCA

MAESTRÍA EN CIENCIAS DE LA INGENIERÍA ELÉCTRICA

MÓDULO 1217 — REDES COMPLEJAS — IV COHORTE

PROYECTO INTEGRADOR DEL MÓDULO

ANÁLISIS DE REDES COMPLEJAS

**Caso de estudio: la red de datos
de la Universidad de Cuenca**

177 nodos y 209 enlaces reconstruidos a partir de los 34 diagramas
del informe técnico de topologías y conexiones de red

DOCUMENTO ACADÉMICO — EVALUACIÓN FORMATIVA DEL MÓDULO

Dr. Fabián Astudillo-Salinas — La U que vive la educación

Cuenca — Ecuador

Índice

1	Información General del Proyecto	3
1.1	Presentación	3
1.2	Ficha del proyecto	3
1.3	Objetivos	4
1.4	Alineación con el sílabo	4
2	El Caso de Estudio	6
2.1	La red de datos de la Universidad de Cuenca	6
2.2	Cómo se construyó el grafo	7
2.3	Archivos de datos y diccionario	8
2.3.1	Atributos de nodo	8
2.3.2	Atributos de arista	9
2.4	Carga de los datos	9
3	Fase 1 — Modelado y Caracterización	11
3.1	Problema P1 — Medidas fundamentales	11
3.2	Problema P2 — Modelos nulos y visualización	12
4	Fase 2 — Recorrido y Partición	13
4.1	Problema P3 — BFS y DFS sobre la red	13
4.2	Problema P4 — Comunidades y modularidad	13
5	Fase 3 — Optimización en Redes	15
5.1	Problema P5 — Caminos más cortos	15
5.2	Problema P6 — Flujo máximo y corte mínimo	16
5.3	Problema P7 — Localización de instalaciones	17
6	Fase 4 — Percolación y Robustez	18
6.1	Problema P8 — Percolación de nodos y enlaces	18
6.2	Problema P9 — Propagación de fallos y epidemias	19
6.3	Problema P10 — Diagnóstico de puntos críticos	19
7	Fase 5 — Propuesta de Rediseño	20
8	Entregables, Rúbrica y Condiciones	21
8.1	Entregables	21
8.2	Rúbrica de evaluación	21
8.3	Fecha de entrega	23
A	Valores de Verificación del <i>Pipeline</i>	24
B	Plantilla de Estructura del Informe	25

C	Recursos y Bibliografía	26
C.1	Material del repositorio del módulo	26
C.2	Herramientas	26
C.3	Bibliografía del módulo	26

Capítulo 1

Información General del Proyecto

1.1 Presentación

Este documento define el **proyecto integrador** del módulo de Redes Complejas. A diferencia de las actividades de laboratorio, que ejercitan un algoritmo a la vez sobre redes de ejemplo de tamaño reducido, aquí se trabaja de principio a fin sobre una **red real de infraestructura**: la red de datos de la Universidad de Cuenca, reconstruida a partir del informe técnico «*Diagramas de red final*», que documenta 34 diagramas de topología de los seis campus y de la interconexión MPLS entre ellos.

El proyecto recorre las cinco unidades del sílabo en cinco fases encadenadas: modelado y caracterización (Unidad 1), recorrido y partición (Unidad 2), optimización en redes (Unidad 3), percolación y robustez (Unidad 4) y, como cierre, una propuesta de rediseño argumentada con evidencia cuantitativa (Unidad 5). Cada fase se apoya en los resultados de la anterior, de modo que el producto final no es una colección de ejercicios sino un **informe técnico único** que responde a una pregunta de ingeniería:

🔍 Pregunta que debe responder el proyecto

¿Cuáles son los puntos críticos de la red de datos de la Universidad de Cuenca, qué tan grave sería su falla, y qué intervenciones concretas — acotadas en número y justificadas con métricas — mejorarían más su robustez y su desempeño?

1.2 Ficha del proyecto

Cuadro 1: Ficha general del proyecto integrador

Campo	Detalle
Programa	Maestría en Ciencias de la Ingeniería Eléctrica
Módulo	Redes Complejas (código 1217, 3,5 créditos / 168 horas)
Unidades cubiertas	Unidades 1 a 5 (proyecto integrador)
Docente	Dr. Fabián Astudillo-Salinas
Modalidad	Grupos de 2 o 3 integrantes
Caso de estudio	Red de datos de la Universidad de Cuenca (6 campus)
Datos	red_ucuenca.graphml, red_ucuenca_nodes.csv, red_ucuenca_edges.csv
Herramientas	Julia (≥ 1.9), Python (≥ 3.10) y Gephi

Repositorio	https://github.com/fabianastudillo/ComplexNetworks
Entregables	Informe PDF + repositorio de código reproducible + presentación de 15 minutos
Puntaje	25 puntos (ver rúbrica, tabla 4)
Fecha de entrega	26 de agosto de 2026

1.3 Objetivos

i Objetivo general

Analizar, diagnosticar y proponer mejoras sobre una red de infraestructura real empleando teoría de grafos, medidas de redes complejas, algoritmos de recorrido, partición y optimización, y teoría de percolación, comunicando los resultados en un informe técnico reproducible.

Al finalizar el proyecto, cada grupo será capaz de:

1. Modelar una red real seleccionando de forma justificada el modelo de peso y la función de capacidad adecuados a la pregunta que se quiera responder.
2. Calcular e interpretar medidas estructurales — grado, centralidad de cercanía e intermediación, *clustering*, modularidad — y contrastar la red observada con modelos nulos.
3. Implementar y aplicar algoritmos de recorrido (BFS, DFS), partición (Louvain, *k*-means), camino más corto (Dijkstra, Floyd-Warshall) y flujo máximo (Ford-Fulkerson, Edmonds-Karp) sobre la red del caso.
4. Evaluar la robustez de la red mediante percolación de nodos y enlaces bajo fallos aleatorios y ataques dirigidos, e identificar puntos críticos.
5. Formular una propuesta de rediseño acotada y defenderla con evidencia cuantitativa, comunicándola por escrito y de forma oral.

1.4 Alineación con el sílabo

La tabla 2 relaciona cada fase del proyecto con los contenidos y los resultados de aprendizaje (RA) del módulo.

Cuadro 2: Alineación de las fases del proyecto con el sílabo del módulo

Fase	Tema	Contenidos	RA
1	Modelado y caracterización	1.1–1.4	RA1, RA3
2	Recorrido y partición	2.1–2.4	RA2, RA3
3	Optimización en redes	3.1–3.5	RA2, RA3, RA4
4	Percolación y robustez	4.1–4.4	RA2, RA3, RA4
5	Propuesta de rediseño y cierre	5.1	RA2, RA4

i Resultados de aprendizaje del módulo

RA1. Utilizar la teoría de grafos como herramienta para el diseño de redes. **RA2.** Modelar problemas de redes de computadoras, telecomunicaciones, telefonía móvil y redes eléctricas con teoría de grafos y resolverlos aplicando los algoritmos relacionados. **RA3.** Manejar de forma experta algoritmos y herramientas informáticas relacionadas con la teoría de grafos. **RA4.** Desarrollar modelos, algoritmos, aplicaciones y/o simuladores que ayuden al diseño, al análisis y a la toma de decisiones.

Capítulo 2

El Caso de Estudio

2.1 La red de datos de la Universidad de Cuenca

Según el informe técnico de referencia, la red institucional está diseñada bajo una topología en estrella estructurada jerárquicamente en tres capas:

- **Capa de *core* (núcleo):** enrutamiento de alta velocidad e interconexión central del campus.
- **Capa de agregación (distribución):** consolidación de los enlaces provenientes de las facultades y bloques administrativos.
- **Capa de acceso:** conectividad directa a los equipos de usuario final y sub-laboratorios.

Los enlaces troncales entre capas operan a 10 Gbps. Los campus **Balzay** y **Paraíso** implementan redundancia física completa en los enlaces de 10 Gbps entre *core* y agregación; el **Campus Central**, en cambio, dispone de enlaces simples entre las capas de agregación y acceso. Los seis campus se interconectan a través de la nube **MPLS** de un proveedor, con enlaces redundantes hacia Campus Central y Campus Balzay.

⚠ Esta asimetría no es un detalle menor

Que un campus tenga redundancia y otro no es, en lenguaje de redes complejas, la diferencia entre un enlace ordinario y un *punte*: una arista cuya eliminación desconecta el grafo. Buena parte del proyecto consiste en cuantificar esa diferencia.

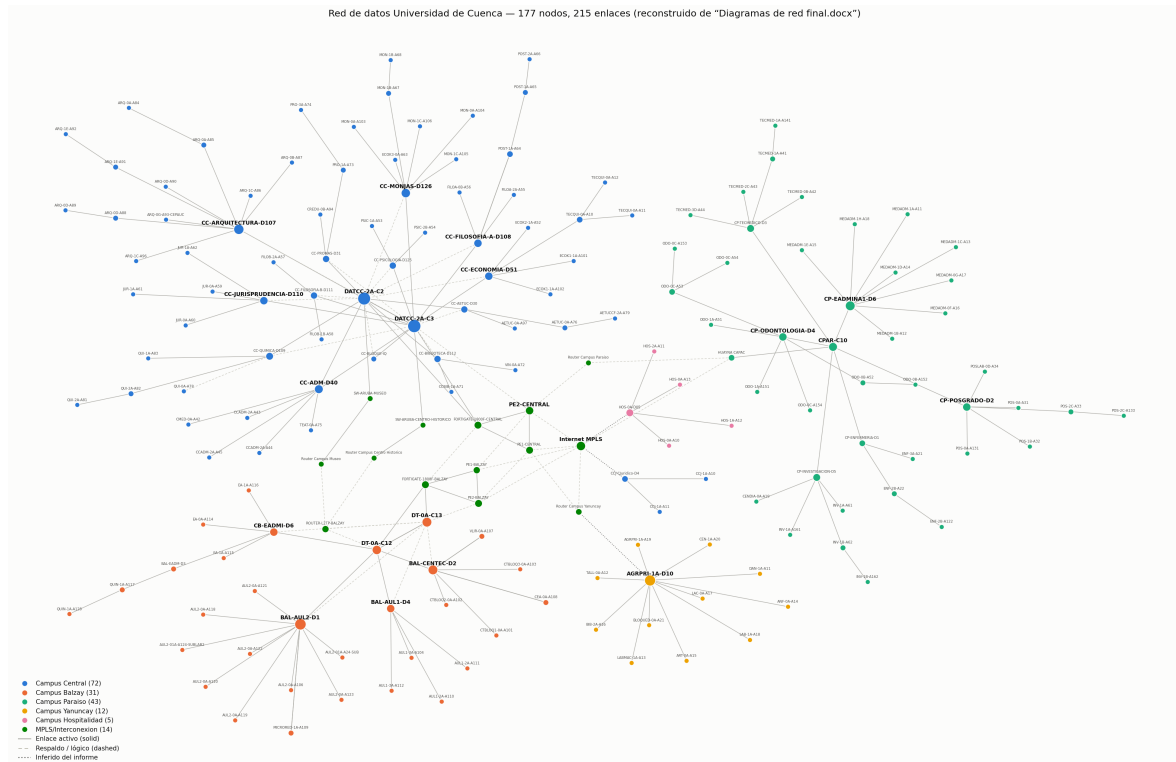


Figura 1: Visualización del grafo reconstruido de la red de datos de la Universidad de Cuenca. El color de los nodos corresponde al campus. Esta figura es solo de referencia: cada grupo debe producir sus propias visualizaciones.

2.2 Cómo se construyó el grafo

Cada una de las 34 imágenes del documento «*Diagramas de red final*» (*weathermap* de monitoreo más el diagrama MPLS) fue transcrita a una lista de nodos y aristas, fusionada en un único grafo no dirigido y posteriormente **saneada**. Las decisiones de reconstrucción relevantes fueron:

1. Los nombres de edificio del diagrama general de cada campus se unificaron con el *switch* de distribución de su diagrama de detalle (p. ej. «CC-ARQUITECTURA» → CC-ARQUITECTURA-D107; «AULARIO 1» → BAL-AUL1-D4; «A4 ENFERMERIA» → CP-ENFERMERIA-D1; «C12» del diagrama MPLS → DT-0A-C12).
2. «Huayna Cápac» (diagrama 27) y el *router* Campus Huayna Cápac (diagrama 34) se trataron como un mismo nodo.
3. Tres enlaces marcados con *rol=inferido* (diagrama 0) conectan Yanuncay, Hospitalidad y el Consultorio Jurídico a la nube MPLS: el informe indica esa interconexión, pero los diagramas no dibujan explícitamente el enlace *router*↔*switch* de núcleo.
4. Seis pares de aristas que parecían enlaces redundantes resultaron ser *el mismo enlace físico transcrito desde dos diagramas distintos*, con dos lecturas de tráfico

diferentes. Se fusionaron conservando la medición máxima, de modo que el grafo es **simple**: 177 nodos y 209 aristas.

i Conjunto de datos saneado

La transcripción original contenía inconsistencias heredadas del documento fuente: identificadores en dos convenciones, aristas duplicadas, un atributo que mezclaba tráfico con capacidad y un atributo de estilo con dos significados a la vez. Todas fueron corregidas antes de entregar el conjunto de datos; `red_ucuenca_correcciones.py` documenta y reproduce cada corrección, y `red_ucuenca_README.md` las resume. **No es necesario auditar los datos**: el grafo que reciben es consistente y está listo para analizarse.

2.3 Archivos de datos y diccionario

Cuadro 3: Archivos del conjunto de datos, en la raíz del repositorio

Archivo	Contenido
<code>red_ucuenca.graphml</code>	Grafo completo con atributos. Importable en Gephi, yEd, Cytoscape, NetworkX, igraph y Graphs.jl (vía GraphIO.jl).
<code>red_ucuenca_nodes.csv</code>	177 nodos: id, label, campus, capa, diagrams.
<code>red_ucuenca_edges.csv</code>	209 aristas: source, target, label, trafico_mbps, capacidad_mbps, rol, diagrams.
<code>red_ucuenca_grafo.png</code>	Visualización de referencia (figura 1).
<code>red_ucuenca_noc.html</code>	Vista interactiva tipo NOC del grafo.
<code>red_ucuenca_README.md</code>	Notas de reconstrucción y correcciones.
<code>red_ucuenca_correcciones.py</code>	Script que produjo la versión saneada a partir de la transcripción original.

2.3.1 Atributos de nodo

- `id` y `label`: nombre del equipo tal como aparece en el diagrama, en una única convención (mayúsculas y guion medio). Los identificadores son **idénticos** en los CSV y en el GraphML.
- `campus`: ubicación física, con un solo valor. Campus Central, Balzay, Yanuncay, Hospitalidad o Paraíso; sedes Museo y Centro Histórico; Nube MPLS.
- `capa`: posición en la jerarquía descrita por el informe: core, agregacion, acceso, wan o interconexion (los *firewalls* perimetrales).
- `diagrams`: número(s) del diagrama de origen; el diagrama N corresponde a la imagen N del documento fuente.

2.3.2 Atributos de arista

- `label`: etiqueta original del enlace (puerto o *lag* y lectura del monitor).
- `trafico_mbps`: tráfico **medido** en el instante de la captura, en Mbps. Definido en 170 de las 209 aristas.
- `capacidad_mbps`: capacidad **nominal** declarada explícitamente en la etiqueta del enlace (10 Gbps, 2×10 Gbps, 1 Gbps). Definida en 28 aristas, todas del diagrama MPLS.
- `rol`: función del enlace. principal (157), wan (32), respaldo (12, miembro secundario de un par redundante sin tráfico), secundario (5, miembro secundario que sí cursa tráfico) e inferido (3).
- `diagrams`: diagrama(s) de procedencia, separados por ;.

⚠ Tráfico medido ≠ capacidad

Son dos magnitudes distintas y por eso viajan en columnas distintas: no las mezclen. La capacidad nominal solo está documentada en los 28 enlaces del diagrama MPLS; para el resto, en la Fase 3 deberán **estimarla y justificarla** a partir de la información disponible (capa jerárquica del equipo, redundancia observada, los 10 Gbps que el informe declara para los troncales), y discutir cómo esa elección afecta los resultados.

2.4 Carga de los datos

El repositorio incluye un **código base** listo para ejecutar en `proyecto/codigo_base /`, con la misma funcionalidad en los dos lenguajes del módulo: carga de los datos, verificación contra el Anexo A y resumen estructural mínimo. Los CSV y el GraphML usan los mismos identificadores, así que ambas fuentes producen exactamente el mismo grafo.

```
1 cd proyecto/codigo_base
2
3 # Python
4 python3 -m pip install -r requirements.txt
5 python3 cargar_red.py # desde los CSV
6 python3 cargar_red.py --fuente graphml # desde el GraphML
7
8 # Julia
9 julia --project=. -e 'using Pkg; Pkg.instantiate()'
10 julia --project=. cargar_red.jl
```

Listing 2.1: Ejecución del código base

Ambos scripts terminan con código de salida 0 si las trece comprobaciones del anexo pasan y 1 si alguna falla. El código base carga y verifica, pero **no pondera las aristas, no estima capacidades y no calcula ninguna métrica**: eso es precisamente el proyecto. Los fragmentos siguientes muestran el núcleo de cada versión.

```

1 import networkx as nx
2
3 G = nx.Graph(nx.read_graphml("red_ucuenca.graphml")) # 177 nodos, 209 aristas
4
5 print(G.number_of_nodes(), G.number_of_edges())
6 print(nx.number_connected_components(G))
7 print(G.nodes["DATCC-2A-C3"]) # label, campus, capa, diagrams
8 print(G["DATCC-2A-C2"]["DATCC-2A-C3"]) # label, trafico_mbps, capacidad_mbps,
   rol

```

Listing 2.2: Carga en Python con NetworkX (codigo_base/cargar_red.py)

```

1 using CSV, DataFrames, Graphs
2
3 nodos = CSV.read("red_ucuenca_nodos.csv", DataFrame)
4 aristas = CSV.read("red_ucuenca_edges.csv", DataFrame)
5
6 idx = Dict{id => i for (i, id) in enumerate(nodos.id)}
7 g = SimpleGraph(nrow(nodos))
8 for r in eachrow(aristas)
9     add_edge!(g, idx[r.source], idx[r.target])
10 end
11
12 println(nv(g), " nodos, ", ne(g), " aristas")
13 println("componentes: ", length(connected_components(g)))

```

Listing 2.3: Carga en Julia desde los CSV, conservando atributos (codigo_base/cargar_red.jl)

GraphML en Julia

GraphIO.jl carga la *estructura* del GraphML pero descarta los atributos. Si necesitan campus, capa, trafico_mbps o rol en Julia, partan de los CSV como en el listado anterior, o construyan un MetaGraph añadiendo los atributos a mano.

Verificación del *pipeline*

Antes de calcular nada, comprueben que su carga reproduce los valores del Anexo A; la función `verificar` del código base lo hace automáticamente. Si parten de cero en lugar de usar el código base, incluyan una verificación equivalente en su propio *pipeline*.

Capítulo 3

Fase 1 — Modelado y Caracterización

Unidad 1 del sílabo — Introducción a redes complejas y teoría de grafos aplicada.
Contenidos 1.1 a 1.4. Peso: 5 puntos.

3.1 Problema P1 — Medidas fundamentales

☰ P1 — Caracterización estructural 3 puntos

Calculen e **interpreten** (no basta con tabular):

1. Número de nodos y aristas, densidad, número de componentes conexas y tamaño de la mayor.
2. Distribución de grado: histograma, grado medio, máximo y mínimo, y gráfico en escala log-log. ¿Se observa una cola pesada? Discutan si tiene sentido hablar de red libre de escala en una red de infraestructura de este tamaño y qué prueba estadística usarían antes de afirmarlo.
3. Centralidad de grado, de intermediación (*betweenness*), de cercanía (*closeness*) y de vector propio. Presenten el **top-10** de cada una en una única tabla comparativa.
4. Coeficiente de *clustering* medio, diámetro, distancia media entre pares y asortatividad por grado.
5. Identifiquen los **puntos de articulación** y los **puentes** del grafo, y contabilícenlos por campus y por capa.
6. Contrasten con los datos lo que afirma el informe técnico (capítulo 2): ¿se observa redundancia *core*–agregación en Balzay y en Paraíso? ¿Y en Campus Central? Usen el atributo *capa* para responder.

🔍 Las preguntas que hay que responder, no solo calcular

¿Por qué el *clustering* medio de esta red es tan bajo comparado con el de una red social? ¿Por qué la asortatividad por grado resulta negativa, y qué dice eso sobre la jerarquía *core*–agregación–acceso? ¿Coinciden los nodos más centrales por grado con los más centrales por intermediación? Si no coinciden, ¿qué papel distinto juega cada grupo en la red? Un informe que solo presenta números sin responder estas preguntas obtiene, como máximo, la mitad del puntaje de P1.

3.2 Problema P2 — Modelos nulos y visualización

☰ P2 — Comparación con modelos nulos y visualización 2 puntos

1. Generen **100** realizaciones de un grafo aleatorio de Erdős–Rényi $G(n, m)$ y **100** de un modelo de configuración que preserve la secuencia de grados observada. Comparen — con media y desviación estándar — el *clustering*, la distancia media, el diámetro y la asortatividad frente a los valores de la red real. ¿Qué propiedades de la red UCuenca *no* se explican por su secuencia de grados?
2. Generen también una red de Barabási–Albert con n y m comparables y discutan por qué una red de infraestructura física se parece o no a un modelo de crecimiento preferencial.
3. Produzcan al menos **dos** visualizaciones propias en Gephi (o en Julia/Python) con criterios distintos: una coloreada por campus y otra con el tamaño de nodo proporcional a la intermediación. Indiquen qué algoritmo de disposición (*layout*) usaron y por qué.

i Entregable de la Fase 1

Sección del informe con las tablas de medidas, el contraste con lo que afirma el informe técnico, las comparaciones con modelos nulos y las dos visualizaciones, más los *scripts* que las generan.

Capítulo 4

Fase 2 — Recorrido y Partición

Unidad 2 del sílabo — Algoritmos en redes complejas. Contenidos 2.1 a 2.4. Peso: 5 puntos.

4.1 Problema P₃ — BFS y DFS sobre la red

☰ P₃ — Recorridos y su lectura operativa 2,5 puntos

1. Implementen BFS y DFS **desde cero** (no usen la función de biblioteca; sí pueden usarla para verificar). Documenten la estructura de datos empleada y la complejidad obtenida.
2. Ejecuten BFS desde el *switch* de *core* del Campus Central y construyan el **perfil de profundidad**: número de nodos a distancia 1, 2, 3, ... Interpreten el resultado en términos de las tres capas jerárquicas del informe. ¿La jerarquía declarada se refleja en las distancias medidas?
3. Repitan el BFS desde el nodo de la nube MPLS y comparen el perfil. ¿Qué campus quedan más «lejos» del resto de la institución?
4. Usen DFS para detectar los **ciclos** del grafo. ¿Dónde están? Relacionen su ubicación con los enlaces redundantes descritos en el informe: la ausencia de ciclos en una zona equivale a ausencia de camino alternativo.
5. Un técnico debe visitar físicamente todos los armarios de un campus partiendo del *core*. Argumenten cuál de los dos recorridos modela mejor esa tarea y por qué.

4.2 Problema P₄ — Comunidades y modularidad

☰ P₄ — Partición de la red 2,5 puntos

1. Apliquen **Louvain** sobre el grafo. Reporten el número de comunidades y la modularidad Q . Repitan la ejecución con al menos cinco semillas distintas y discutan la estabilidad del resultado.
2. Comparen la partición obtenida con la partición «natural» por campus usan-

do **información mutua normalizada (NMI)** y el **índice de Rand ajustado (ARI)**. Presenten además una matriz de confusión comunidad $\times$ campus.

3. Identifiquen los nodos donde ambas particiones *discrepan*. ¿Qué significa, en términos de ingeniería de red, que un equipo etiquetado en un campus quede agrupado con otro?
4. Apliquen *k-means* sobre una representación vectorial de los nodos (por ejemplo, las primeras componentes del *espectro* del laplaciano, o un *embedding* tipo *node2vec*). Comparen el resultado con Louvain y discutan por qué un método basado en distancias euclídeas puede o no ser adecuado para un grafo.
5. Discutan la **limitación de resolución** de la modularidad: ¿podría Louvain estar fusionando bloques que un administrador de red consideraría separados? Aporten evidencia.

Reutilicen el código del módulo

El repositorio incluye `algoritmos/kmeans/` y el material de `intro/codes/` (incluido `gen_visualizacion-louvain.jl`). Reutilizar y adaptar ese código es correcto y esperado; citar de dónde proviene cada pieza es obligatorio.

Capítulo 5

Fase 3 — Optimización en Redes

Unidad 3 del sílabo — Optimización en redes complejas. Contenidos 3.1 a 3.5. Peso: 6 puntos.

5.1 Problema P5 — Caminos más cortos

Antes de aplicar cualquier algoritmo de camino más corto hay que decidir **qué se minimiza**. Tres modelos de peso razonables sobre esta red son:

$$\begin{aligned}w_{\text{saltos}}(u, v) &= 1 \\w_{\text{latencia}}(u, v) &= \alpha + \frac{\beta}{c(u, v)} \\w_{\text{carga}}(u, v) &= \frac{b(u, v)}{c(u, v)}\end{aligned}$$

donde $c(u, v)$ es la capacidad del enlace, $b(u, v)$ el tráfico medido que registra el atributo `trafico_mbps`, y α y β son constantes que cada grupo debe fijar y justificar.

☰ P5 — Dijkstra y Floyd-Warshall 2 puntos

1. Implementen **Dijkstra** con cola de prioridad y **Floyd-Warshall**. Verifiquen que coinciden sobre la red UCuenca para al menos 20 pares de nodos elegidos al azar.
2. Comparen empíricamente sus tiempos de ejecución sobre la red completa y sobre subredes de tamaño creciente. Contrasten la curva medida con las complejidades teóricas $O((V+E) \log V)$ y $O(V^3)$: ¿a partir de qué tamaño, y para cuántos pares consultados, conviene cada uno?
3. Calculen la matriz de distancias completa con los tres modelos de peso. ¿Cambia el ranking de nodos por cercanía según el peso elegido? Presenten los tres *top-10* lado a lado.
4. Determinen el par de equipos de acceso **más distante** de la institución bajo cada modelo de peso y describan la ruta correspondiente salto a salto.
5. Discutan por qué un protocolo real (OSPF, IS-IS) usaría uno u otro modelo de peso, y qué le ocurriría a la red si el peso dependiera del tráfico instantáneo.

⚠ Pesos negativos

Si alguno de sus modelos de peso puede producir valores negativos, Dijkstra deja de ser válido. Justifiquen que sus pesos son no negativos o cambien de algoritmo (Bellman-Ford) y explíquenlo en el informe.

5.2 Problema P6 — Flujo máximo y corte mínimo

☰ P6 — Capacidad y cuellos de botella 2,5 puntos

1. **Completen la función de capacidad** $c(u, v)$. El atributo `capacidad_mbps` la da directamente para los 28 enlaces del diagrama MPLS; para los 181 restantes deberán estimarla a partir de la información disponible (atributo `capa` de los extremos, `rol` del enlace, los 10 Gbps que el informe declara para los troncales). Documenten cada supuesto; se evalúa la *justificación*, no la cifra concreta.
2. Modelen el problema «todo el tráfico de un campus sale hacia Internet»: fuente s = agregación de los equipos de acceso del campus (usen un super-nodo), sumidero t = nodo de salida a Internet/MPLS. Calculen el flujo máximo con **Ford-Fulkerson** y con **Edmonds-Karp**, reutilizando las implementaciones de `optimization/ford-fulkerson/` y `optimization/edmonds-karp/`.
3. Reporten para cada campus: flujo máximo, número de iteraciones de cada algoritmo, longitudes de los caminos aumentantes y el **corte mínimo** obtenido. Verifiquen *a mano* la capacidad de al menos un corte mínimo sumando las capacidades de sus aristas.
4. Interpreten el corte mínimo: ¿qué enlaces físicos concretos lo componen? ¿Coinciden con los puentes hallados en P1? ¿Coinciden con los enlaces que el informe describe como no redundantes?
5. Formulen un problema de **flujo de costo mínimo** sobre la misma red: asignen un costo por unidad de flujo a cada enlace (por ejemplo, proporcional al número de saltos o al uso de un enlace de respaldo) y resuelvan el envío de una demanda fija desde dos campus hacia Internet. Comparen la solución con la de flujo máximo puro.

5.3 Problema P7 — Localización de instalaciones

☰ P7 — ¿Dónde colocar los recursos? 1,5 puntos

La institución quiere instalar p colectores de telemetría (*NetFlow*, *SNMP*) de modo que ningún equipo quede demasiado lejos de uno.

1. Formulen el problema como un p -**mediana** (minimizar la distancia media a la instalación más cercana) y como un p -**centro** (minimizar la distancia máxima). Escriban ambos modelos matemáticamente.
2. Resuélvanlos para $p \in \{1, 2, 3, 5\}$ con una heurística voraz y, si les es posible, con un solver de programación entera (JuMP en Julia, PuLP o *scipy* en Python).
3. Comparen las ubicaciones obtenidas con los nodos de mayor intermediación y de mayor cercanía de P1. ¿Coinciden? ¿Por qué la respuesta a «dónde poner un colector» no es necesariamente «en el nodo más central»?
4. Discutan qué restricción práctica omite el modelo (energía, espacio en rack, seguridad física, costo de licencias) y cómo la incorporarían.

Capítulo 6

Fase 4 — Percolación y Robustez

Unidad 4 del sílabo — Teoría de percolación y robustez en redes. Contenidos 4.1 a 4.4.
Peso: 6 puntos.

6.1 Problema P8 — Percolación de nodos y enlaces

Sea $S(f)$ el tamaño relativo de la componente conexa gigante tras eliminar una fracción f de los nodos (percolación de sitios) o de las aristas (percolación de enlaces). El **umbral de percolación** f_c es el valor de f en el que la componente gigante colapsa.

☰ P8 — Fallos aleatorios frente a ataques dirigidos 2,5 puntos

1. Implementen percolación de **nodos** bajo cuatro estrategias: (a) fallo aleatorio, (b) ataque por grado descendente, (c) ataque por intermediación descendente, y (d) ataque por intermediación **recalculada** tras cada eliminación. Para (a), promedien al menos 100 realizaciones y reporten la desviación estándar.
2. Grafiquen $S(f)$ frente a f para las cuatro estrategias en una sola figura. Estimen f_c en cada caso y expliquen las diferencias.
3. Repitan el análisis para percolación de **enlaces**, incluyendo un ataque dirigido a los puentes identificados en P1.
4. Reporten además la **eficiencia global** de la red $E = \frac{1}{n(n-1)} \sum_{i \neq j} 1/d_{ij}$ en función de f , y discutan por qué puede degradarse mucho antes de que la componente gigante se rompa.
5. Comparen la curva de la red UCuenca con la de los modelos nulos de P2 (Erdős-Rényi y configuración). ¿Es esta red más o menos robusta de lo que su secuencia de grados haría esperar?

🔍 La paradoja de la robustez

Las redes con distribución de grado heterogénea suelen ser muy resistentes a fallos aleatorios y muy frágiles frente a ataques dirigidos. Verifiquen si esta red exhibe ese comportamiento y, sobre todo, **expliquen la consecuencia operativa**: ¿qué implica para un plan de mantenimiento, y qué para un plan de respuesta ante incidentes de seguridad?

6.2 Problema P9 — Propagación de fallos y epidemias

☰ P9 — Modelos dinámicos sobre la red 2 puntos

1. Implementen un modelo de **propagación de fallos en cascada**: asignen a cada nodo una carga inicial proporcional a su intermediación y una capacidad $C_i = (1 + \tau)L_i$ con margen de tolerancia τ . Al eliminar un nodo, redistribuyan su carga por los caminos más cortos; los nodos que superen su capacidad fallan y la cascada continúa.
2. Determinen el **margen crítico** τ_c por debajo del cual la falla de un único nodo provoca una cascada que afecta a más del 20 % de la red. Identifiquen los nodos disparadores más peligrosos.
3. Simulen un modelo **SIR** sobre la red (interpretación: un *malware* que se propaga entre equipos, o un fallo de configuración replicado). Estimen el umbral epidémico y compárenlo con la predicción de campo medio $\lambda_c \approx \langle k \rangle / \langle k^2 \rangle$.
4. Evalúen dos estrategias de **inmunización** con el mismo presupuesto (parchear m equipos): al azar frente a por centralidad. Cuantifiquen la reducción del tamaño final del brote.
5. Relacionen los resultados con el caso análogo en un **sistema de energía eléctrica**: ¿qué corresponde a la carga, a la capacidad y a la cascada en una red de transmisión? Citen al menos una referencia de la bibliografía del módulo.

6.3 Problema P10 — Diagnóstico de puntos críticos

☰ P10 — Síntesis del diagnóstico 1,5 puntos

Consoliden los resultados de las Fases 1 a 4 en un **ranking único de puntos críticos** de la red. Definan y justifiquen un índice compuesto (por ejemplo, una combinación normalizada de intermediación, condición de punto de articulación, participación en el corte mínimo y daño causado en la cascada). Presenten el top-10 con una ficha por nodo: identificador, campus, función en la jerarquía, métricas que lo señalan y consecuencia estimada de su falla.

Capítulo 7

Fase 5 — Propuesta de Rediseño

Unidad 5 del sílabo — Aplicaciones. Contenido 5.1. Peso: 3 puntos.

☰ P11 — Intervención acotada y justificada 3 puntos

Con el diagnóstico de P10, propongan una intervención sobre la red sujeta a una restricción dura: **a lo sumo cinco enlaces nuevos** (o cinco cambios de topología, contando la duplicación de un enlace existente como uno).

1. Describan cada intervención propuesta: qué nodos une, qué capacidad asume, qué problema del diagnóstico resuelve.
2. **Cuantifiquen la mejora** recalculando sobre la red modificada, como mínimo: número de puentes y puntos de articulación, distancia media, eficiencia global, curva de percolación bajo ataque dirigido y flujo máximo por campus. Presenten una tabla «antes / después / variación».
3. Justifiquen que su propuesta es **mejor que las alternativas**: comparen contra al menos dos conjuntos alternativos de cinco enlaces (por ejemplo, el que un criterio ingenuo de «unir los dos nodos de mayor grado» produciría).
4. Estimen de forma cualitativa el costo y la factibilidad de la intervención (distancia física entre edificios, necesidad de obra civil, puertos disponibles). No se pide un presupuesto: se pide honestidad sobre qué tan realizable es lo que proponen.
5. Cierren con las **limitaciones** del estudio: qué no captura el modelo, qué datos harían falta y qué conclusiones *no* pueden extraerse de este análisis.

☑ Criterio de excelencia

Un proyecto sobresaliente no es el que aplica más algoritmos, sino el que conecta cada resultado con una decisión de ingeniería. La pregunta que debe poder responder la última página del informe es: «si usted fuera el administrador de esta red y tuviera presupuesto para cinco enlaces, ¿cuáles construiría y por qué?».

Capítulo 8

Entregables, Rúbrica y Condiciones

8.1 Entregables

1. **Informe técnico en PDF**, máximo 30 páginas sin contar anexos, con portada, resumen ejecutivo de una página, una sección por fase, conclusiones y referencias en formato APA. Debe incluir los nombres y correos institucionales de todos los integrantes.
2. **Repositorio de código** en GitHub, con:
 - `README.md` que explique cómo reproducir todos los resultados;
 - `Project.toml/Manifest.toml` (Julia) o `requirements.txt/environment.yml` (Python);
 - *scripts* numerados por fase, ejecutables de principio a fin sin intervención manual;
 - historial de *commits* que evidencie el trabajo de todos los integrantes.
3. **Figuras y archivos derivados**: visualizaciones, curvas de percolación, archivo de proyecto de Gephi si lo usaron.
4. **Presentación oral** de 15 minutos más 5 de preguntas, con diapositivas entregadas en PDF.

! Reproducibilidad

Un resultado que no se puede regenerar ejecutando el repositorio no cuenta como resultado. El docente ejecutará los *scripts*; si un número del informe no aparece al ejecutarlos, ese apartado se califica sobre cero.

8.2 Rúbrica de evaluación

Cuadro 4: Rúbrica de evaluación del proyecto (sobre 25 puntos)

Fase	Criterio de calidad	Detalle	Puntos
Fase 1 (P1–P2)	Medidas correctas e <i>interpretadas</i> ; contraste con lo que afirma el informe técnico; comparación con modelos nulos; visualizaciones legibles y explicadas.	P1: 3,0 P2: 2,0	5
Fase 2 (P3–P4)	BFS y DFS implementados y verificados; lectura operativa de los perfiles de profundidad; Louvain con análisis de estabilidad; comparación cuantitativa con la partición por campus; discusión de la limitación de resolución.	P3: 2,5 P4: 2,5	5
Fase 3 (P5–P7)	Dijkstra y Floyd-Warshall implementados y contrastados; modelos de peso justificados; capacidad estimada y argumentada; flujo máximo y corte mínimo verificados; modelos de localización correctamente formulados.	P5: 2,0 P6: 2,5 P7: 1,5	6
Fase 4 (P8–P10)	Percolación con las cuatro estrategias y promediado adecuado; cascadas y SIR implementados; umbral crítico estimado; comparación con modelos nulos; ranking de puntos críticos justificado.	P8: 2,5 P9: 2,0 P10: 1,5	6
Fase 5 (P11)	Propuesta acotada, cuantificada y comparada con alternativas; limitaciones reconocidas con honestidad.	P11: 3,0	3
Total			25

Cuadro 5: Ajustes transversales sobre la nota técnica

Aspecto	Descripción	Ajuste
Reproducibilidad	Repositorio que se ejecuta de principio a fin sin intervención manual.	hasta –20 %
Calidad de la redacción	Estructura, ortografía, figuras con leyenda y referencia en el texto, citas en APA.	hasta –10 %
Presentación oral	Claridad, dominio del contenido, respuesta a preguntas, participación equilibrada del grupo.	±10 %
Hallazgos originales	Inconsistencias del conjunto de datos correctamente documentadas, análisis no solicitados pero pertinentes.	hasta +5 %

⚠ Integridad académica

El trabajo es grupal y cada grupo entrega un informe propio. Se permite discutir ideas entre grupos, pero **no** compartir texto, código ni resultados. Todo material externo utilizado — incluidos fragmentos de código y salidas de herramientas de inteligencia artificial generativa — debe citarse explícitamente indicando qué se usó y para qué.

8.3 Fecha de entrega

! Entrega: martes 26 de agosto de 2026

Todos los entregables de la sección 8.1 — informe PDF, repositorio de código, figuras y diapositivas — se entregan el **26 de agosto de 2026**. La presentación oral se realiza en la sesión acordada para esa fecha.

Se recomienda avanzar por fases y llevar el informe al día en lugar de dejar todo el análisis para los últimos días: las Fases 4 y 5 dependen de los resultados de las anteriores y no pueden completarse sin ellos.

Capítulo A

Valores de Verificación del *Pipeline*

Los valores de la tabla 6 corresponden **únicamente** a la carga y a las propiedades básicas del grafo, y se publican para que cada grupo confirme que su *pipeline* de lectura de datos es correcto antes de empezar a analizar. Los resultados de centralidades, comunidades, flujos y percolación **no** se publican: forman parte de lo que hay que obtener.

Cuadro 6: Valores de referencia para verificar la carga de los datos

Propiedad	Valor
Nodos	177
Aristas	209
Componentes conexas	1
Densidad	0,0134
Nodos con capa=acceso	132
Nodos con capa=agregacion	27
Nodos con capa=core	5
Nodos con capa=wan	11
Nodos con capa=interconexion	2
Aristas con rol=principal	157
Aristas con rol=wan	32
Aristas con rol=respaldo	12
Aristas con rol=secundario	5
Aristas con rol=inferido	3
Aristas con trafico_mbps definido	170
Aristas con capacidad_mbps definido	28

Cuadro 7: Distribución de nodos por ubicación

Ubicación	Nodos
Campus Central	75
Campus Paraíso	44
Campus Balzay	35
Campus Yanuncay	13
Campus Hospitalidad	5
Sede Museo	2
Sede Centro Histórico	2
Nube MPLS	1
Total	177

Capítulo B

Plantilla de Estructura del Informe

Se sugiere la siguiente estructura, que puede adaptarse siempre que se cubran todos los apartados:

1. **Portada** — título, integrantes, correos institucionales, fecha, enlace al repositorio.
2. **Resumen ejecutivo** (1 página) — problema, método, tres hallazgos principales y la recomendación final.
3. **Introducción** — el caso de estudio y las preguntas que se responden.
4. **Caracterización estructural** — Fase 1: medidas, contraste con el informe técnico, modelos nulos y visualizaciones.
5. **Recorridos y comunidades** — Fase 2.
6. **Optimización** — Fase 3: caminos mínimos, flujos y localización.
7. **Robustez** — Fase 4: percolación, cascadas, epidemias y ranking de puntos críticos.
8. **Propuesta de rediseño** — Fase 5, con la tabla «antes / después».
9. **Conclusiones y limitaciones.**
10. **Referencias** en formato APA.
11. **Anexos** — tablas extensas, código relevante, figuras adicionales.

Sobre las figuras

Toda figura debe tener leyenda, ejes rotulados con unidades y estar referenciada desde el texto. Una curva de percolación sin indicar qué estrategia representa cada línea, o un grafo sin explicar qué codifican el color y el tamaño de los nodos, no aporta evidencia y no se puntúa.

Capítulo C

Recursos y Bibliografía

C.1 Material del repositorio del módulo

Cuadro 8: Material reutilizable del repositorio del módulo

Ruta	Contenido
proyecto/codigo_base/	Punto de partida del proyecto: <code>cargar_red.py</code> y <code>cargar_red.jl</code> (carga, normalización de identificadores, verificación del Anexo A y resumen estructural), más <code>Project.toml</code> y <code>requirements.txt</code> .
intro/codes/	Ejemplos de carga y visualización de grafos, distribución de grado, redes libres de escala y Louvain en Julia.
algoritmos/kmeans/	Implementación de k -means con ejemplos.
optimization/ford-fulkerson/	Ford-Fulkerson con búsqueda BFS o DFS, corte mínimo y animaciones paso a paso.
optimization/edmonds-karp/	Edmonds-Karp con registro de niveles BFS y animación de la onda BFS.
optimization/actividad/	Actividad de laboratorio sobre flujo máximo, base directa del Problema P6.

C.2 Herramientas

- **Julia** ≥ 1.9 con `Graphs.jl`, `MetaGraphsNext.jl`, `GraphIO.jl`, `CSV.jl`, `DataFrames.jl`, `Plots.jl`, `JuMP.jl`.
- **Python** ≥ 3.10 con `networkx`, `numpy`, `scipy`, `pandas`, `matplotlib`, `python-louvain` o `networkx.algorithms.community`.
- **Gephi** para visualización y exploración interactiva.
- **Git** y **GitHub** para el control de versiones del proyecto.

C.3 Bibliografía del módulo

Bibliografía básica

Newman, M. E. J. (2010). *Networks: An Introduction*. Oxford University Press.

Deo, N. (2017). *Graph Theory with Applications to Engineering and Computer Science*. Dover Publications.

Latora, V., Nicosia, V., & Russo, G. (2017). *Complex Networks: Principles, Methods and Applications*. Cambridge University Press.

Bibliografía complementaria

Bondy, J. A., & Murty, U. S. R. (1976). *Graph Theory with Applications*. Macmillan.

West, D. B. (2001). *Introduction to Graph Theory* (2.^a ed.). Prentice Hall.

Artículos científicos que se proveerán durante el curso.

Fuente primaria del caso de estudio

Universidad de Cuenca. *Informe técnico: topologías y conexiones de red* («Diagramas de red final»), 34 diagramas de los campus Central, Balzay, Yanuncay, Hospitalidad y Paraíso, y de la interconexión MPLS.